

~~Memory~~ OS theory: 4 (4.4), 6, 7, 8, 9

Process = running program

CPU virtualization → looks like there are more CPUs, when there is just one.

↳ Time sharing → as many concurrent processes as you want

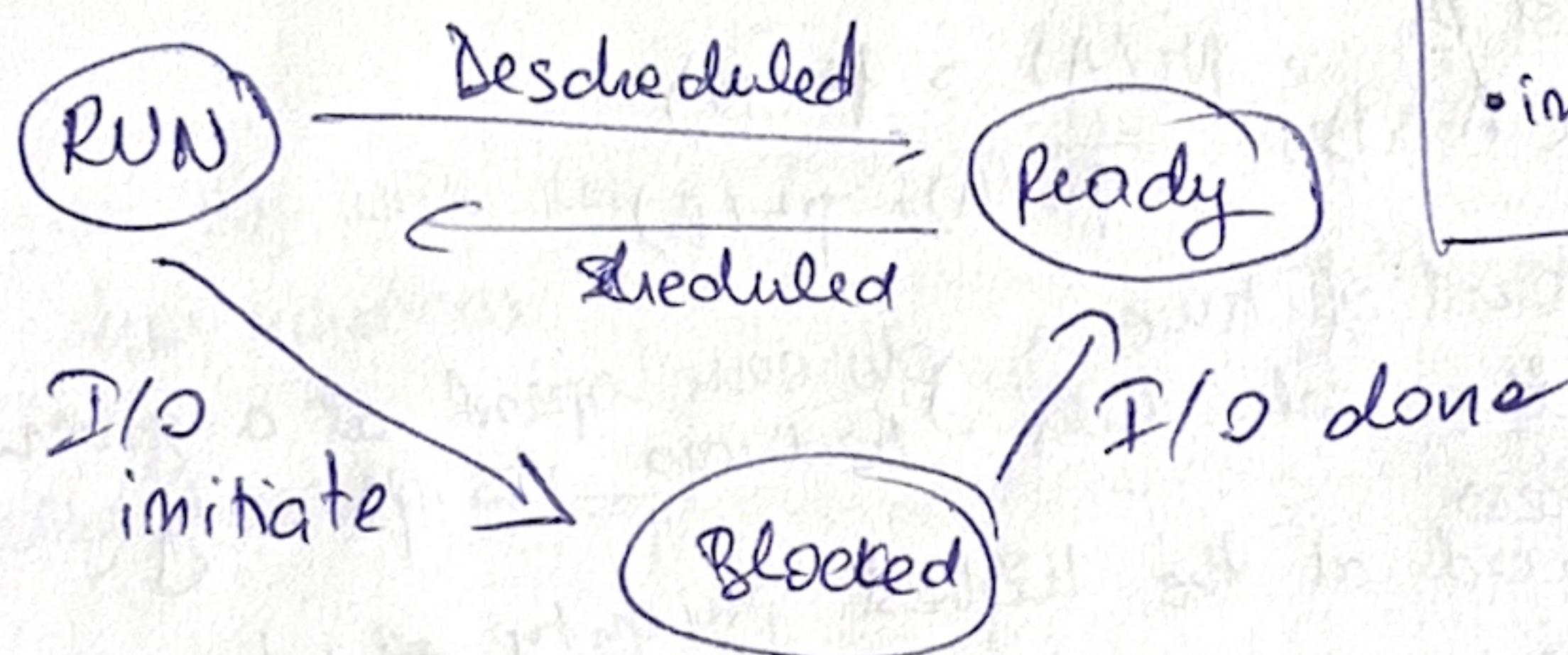
⊖ performance.

→ LOW-LEVEL MACHINERY = MECHANISMS

→ HIGH-LEVEL POLICIES

PROCESS ^{at a point in time, STATE}

- contents of memory in its address space
- contents of CPU registers
 - program counter
 - stack pointer
- info abt. I/O



Mechanisms 6

Time sharing: run one pr. for a little while, then run another one, ...
⇒ VIRTUALIZATION.

Limited direct execution
↓
run on CPU.

Scheduling 7

4 assumptions
→ relax one & identify problems with each scheduling policy

1) FIFO/FIFS

⊕ simple

⊕ easy

⊖ conway effect = a nr. of relatively short potential consumers of a resource get queued behind a heavy weight resource consumer.

2) SJF (shortest job first) ⊕ good (optimal) if all jobs arrive at the same time
→ run the shortest job first, then the next shortest ...

⊖ conway

3) STCF

→ any time a new job enters the system, STCF finds which remaining job's has the least time left ⇒ schedule that one.

4) RR/Robin

→ run a job for a time slice (scheduling quantum) → switch to the next job in the run queue. (ex: timer interrupts every 10 ms ⇒ time slice: 10/20/30 ms)
! length of time quantum must be multiple of the timer-interrupt period.

small $d.t.g \Rightarrow$ better performance.

SJF, STCF

- ↓
- ⊕ optimise turnaround time
 - ⊖ bad for response time

RR

- ↓
- ⊕ optimise response time
 - ⊖ bad for turnaround

5) MLFQ

(multi-level-feedback queue)

→ optimise turnaround time

Rule 1: A, not B, $pr(A) > pr(B)$

Rule 2: RR(A & B), $pr(A) = pr(B)$

Allotment = the amount of time a job can spend at a given priority level before the scheduler reduces its priority.

Rule 3: ^{process} placed at the highest priority when it enters the system

Rule 4a: if a job uses up its allotment while running, its priority is reduced (move down the queue)

Rule 4b: if a job gives up CPU before allotment is up, it stays at the same priority level (ex: I/O op.)

⊖ starvation

↳ if $\exists$ too many interactive jobs in the system, they will consume all CPU, so longer jobs won't receive any CPU (starve)

Rule 5: periodically boost priority of all jobs.

6) Lottery → uses randomness in a clever way to achieve proportional share

100 tickets: → winning: $A \in (0, 74)$

A or B ? $B \in (75, 99)$

↙ $63 \in A \Rightarrow A$ runs

↘ $85 \in B \Rightarrow B$ runs,

7) Stride → deterministically lottery (no randomness)

8) FFS → fair-share scheduling.

↳ weighted RR with dyn. time slices.

- ① • non-deadlock bugs $\rightarrow$ atomicity
 $\rightarrow$ ordering

• deadlock bugs

T₁
 - lock(L₁);
 - lock(L₂);

T₂
 - lock(L₂);
 - lock(L₁);

31

Semaphore = an obj. with an integer value that ^{we} can manipulate with two routines $\rightarrow$ sem-wait() $\rightarrow$ P() $\rightarrow$ decrement
 $\rightarrow$ sem-post() $\rightarrow$ V()

The init. value of the semaphore determines its behaviour

sem_init(&s, 0, 1)



sem. shared between threads
 in the same process.

int sem_wait (...) {

decrement the value of semaphore s by one;
 wait if value of semaphore is negative;

}

Binary Semaphore (init w/ 1)

Ordering sem (init w/ 0)

- Producer / Consumer

RWlocks

Exercises.

acquire write lock: `rwlock_acquire_writelock()`.

release write lock: `rwlock_release_writelock()`.

acquire read lock: - reader acquires lock
- increments the readers var. (how many readers inside)

RAID 38

RAID = redundant array of inexpensive disks
↳ technique to use multiple disks in concern to build a faster / bigger / more reliable disk system.

mirrored-RAID → simple, reliable
→ good perf. but at a high-capacity cost.

RAID-5 → reliable, better from a capacity p.o.v.
→ performs poorly when E small writes.

File sys 40

File system = pure software

↳ DS of the file system
↳ Access methods

create file → allocate inode → bitmap search for 0 → set to 1 ...

Metadata = info. abt. each file → ^{only when allocation is needed!} stored in INODE, usually.

Directory = specific type of file that stores name → inode ~~name~~ mappings

Bitmap = used to track which inodes / data blocks are free / allocated

21

Page fault = the act of accessing a page that is not in the phys. memory
The OS page-fault-handler runs to service the page fault.

Segmentation: ① supports sparse address spaces
② fast
③ code-sharing

allocate variable-size segments in memory $\Rightarrow$ external fragmentation
(segments are variable-sized)

Paging. [18]

I. chop into variable-sized pieces $\Rightarrow$ segmentation in virtual memory.
② fragmentation

II. chop into fixed-sized pieces $\Rightarrow$ paging

Physical memory = array of fixed-sized slots = PAGE FRAMES

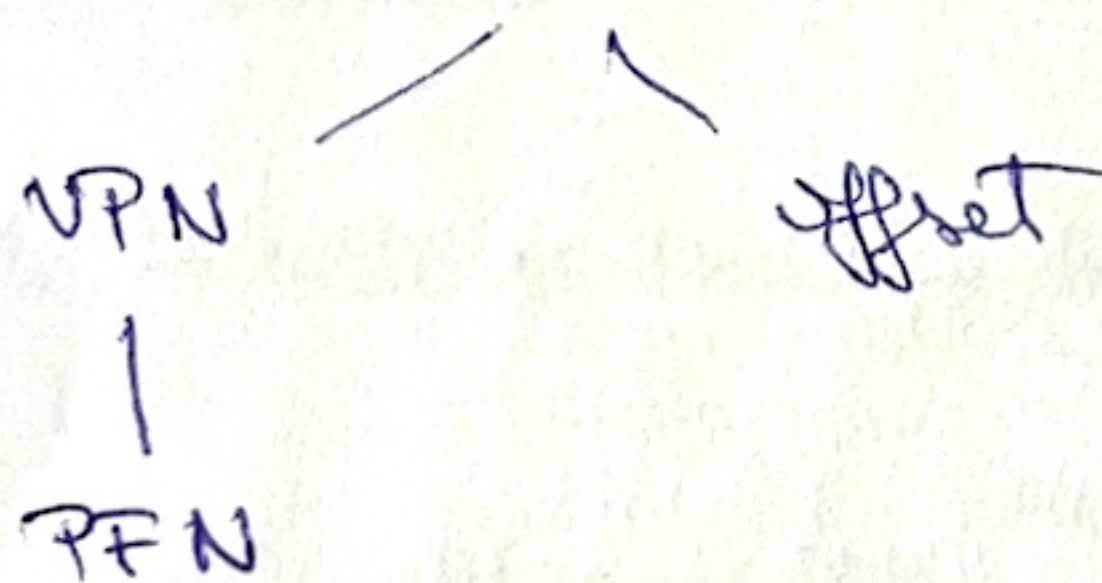
Paging ① flexibility
② simplicity $\rightarrow$ stores virtual-to-physical address transl.

Page table - to record where each virtual page of the address space is placed in the physical memory.
- store address translations for each of the virtual pages of the address space.

III - per-process DS

page / process

Translate virtual address that the process generated:



Page table: map

Virtual addr. $\longleftrightarrow$ physical addr.

$\rightarrow$ DS: Linear page table = array

① does not lead to external fragmentation
(because it chops in fixed-size units)

② slower if not implemented correctly
③ memory waste

fork()

Processes | 5 |

Child - its own copy of the address space
- its own registers
- its own pc

parent receives the Pid of the child
child receives ~~the~~ the ~~Pid of the~~ parent

wait()

- parent calls wait() to block until child finishes execution
- makes output deterministic.

wait() returns in
the parent.



unblock parent

exec()

→ given - the name of an executable
- some args.

} ⇒ it loads code from that
executable and overwrites

⊕

overwrites current code segm.

⊕

heap, stack re-initialised

↳ DOES NOT CREATE a new process, but
it TRANSFORMS the currently running
program into a ≠ running program.

↳ success ⇒ never returns.

Free-space man.

14

mpaging

→ External fragmentation = the free space gets chopped into little pieces
of ≠ sizes.

more focused on.

⇒ A single contiguous space that can satisfy the request,
even though the TOTAL amount of free space exceeds
the size of the request.

Free list = the generic AS that is used to manage free space in the heap.

Internal fragmentation = when an allocator hands out chunks
of memory bigger than requested

Strategies for managing free space:

Best fit → smallest block that can fit requested size.

⊕ heavy performance → exhaustive search

Worst fit → largest chunk → return req. amount → keep remaining
chunk on the free list

⊕ costly

⊕ fragm.

First fit $\rightarrow$ first block big enough

⊕ speed

⊖ holes

Simulate
Performance

use address-based-ordering. Keep list ordered by the addr.
The free space $\Rightarrow$ ⊕ reduced fragm.

⊕ easier coalescing

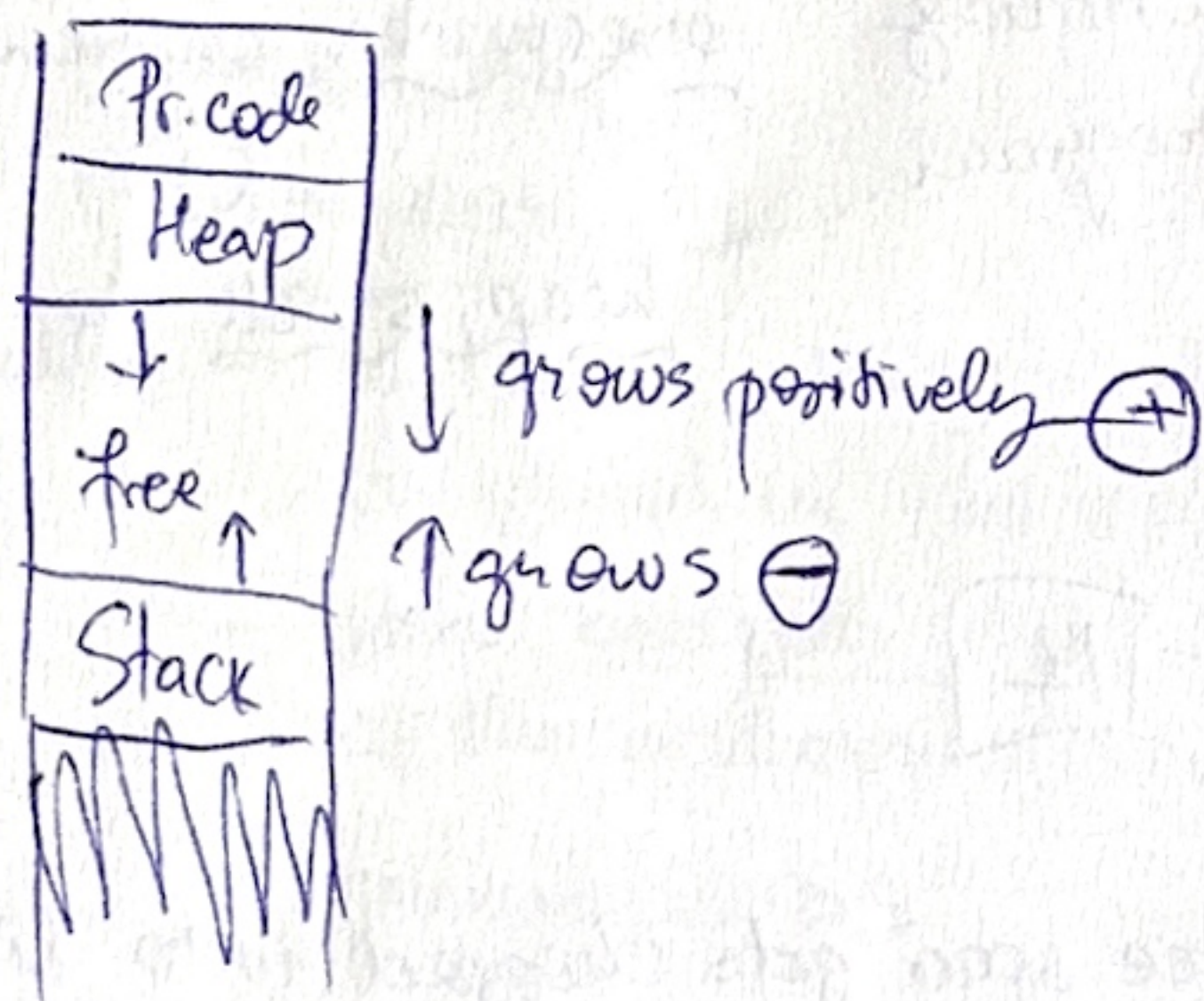
Next fit $\rightarrow$ Keep extra pointers to the location within the list where we were looking last.

$\sim$ first fit performance

Buddy -

13

Address space = abstraction



15

Static relocation = a piece of software (loader) takes code $\rightarrow$ rewrites its address to the desired loc. in the physical memory.

Translation: $P.a = V.a + \text{base}$

dynamic relocation = a base register is used to transform virtual addr.

internal fragmentation = space goes to waste (not all used)

TRANSLATION ↓

Phys. addr.

16

Segmentation fault = access to an illegal address.

External fragmentation = physical memory becomes full of small holes of free space, making it difficult to allocate new segments/grow existing ones

Cond. var. → when some kind of signaling must take place between threads, if one thread is waiting for another to do sth. before it continues.

→ you must have a lock associated with this condition

~~int~~ pthread_cond_wait(&cond, &mu); } → that lock must be held
pthread_cond_signal(&cond); | when calling these.

- wait() → puts the calling thread to sleep
→ waits for other thread to signal it
(usually when sth. in the program has changed)

usage ex:

Thread A
(calling thread)

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;  
  
pthread_mutex_lock(&lock);  
while (ready == 0)  
    pthread_cond_wait(&cond, &lock);  
pthread_mutex_unlock(&lock);
```

Thread B
(waking thread)

```
pthread_mutex_lock(&lock); ← gets A because it sleeps and it's unlocked.  
ready = 1;  
pthread_signal(&cond); → A may continue when the mutex becomes available?  
pthread_mutex_unlock(&lock); ←
```

→ Never signal without having the lock held!
(prevents race conditions)

Why does the wait() call take a lock as its second arg, but the signal() call takes just the condition?

→ because wait(), in addition to putting the calling thread to sleep, ALSO releases the lock when putting that thread to sleep

→ needed this unlock() because otherwise, the awakened thread remains blocked waiting for the mutex

`pthread_join (thread, value);`

↓
which thread
to wait for

↳ pointer to the return value
you expect to get back

ex: `void *res;`
`pthread_join(tid, &res);`

pthread_mutex_lock(&m);
pthread_mutex_unlock(&m); } locks are useful when we have a region of code that is a critical section.

```

u. [ int kc = pthread_mutex_init(&lock, NULL);
      assert(kc == 0);
    ]

```

optional set of
attributes
($NULL = \text{defaults}^{\text{use}}$)

- ↳ returns failure if the lock (m) is already held.
- ↳ should be avoided in general.

If another thread holds the lock, the thread who wants it won't return from the call until it has acquired it.

- ! `-wait()` → unlock mutex → sleep
→ wake up ⇒ lock mutex again before returning
- `signal()` → wake the thread
→ does not transfer ownership of the mutex

! Before returning after being woken, `-wait()` re-acquires the lock, ensuring that any time the waiting thread is running between the lock acquire and the lock release, it holds the lock.

Compile : → header `pthread.h`

link : `-pthread` flag

ex: `prompt> gcc -o main main.c -Wall -pthread`

! Each thread has its own stack

! Don't pass pointers to local (=stack) var.

ex:

```
void f() {
    int x = 5;
    pthread_create(..., &x);
}
```

x gets destroyed here (local) ⇒ WRONG
(a local variable dies when the function gets out of scope)
(stack memory)

↓

```
void f() {
    int *x = malloc(sizeof(int));
    *x = 5;
    pthread_create(&t, NULL, thread_fct, x);
}
```

pointer x gets destroyed, but *x remains allocated
(heap memory) stays alive until `free()`

Cond. Var. [30]

Cond. var = explicit queue that threads can put themselves on when some condition is not as desired, by waiting on the condition.

some other thread, when it changes state, can wake one/more of those waiting threads, by signaling on the condition.

• `PTHREAD_COND_INITIALIZER` ← init.

• `pthread_cond_t c;` ← declare

• `wait()` → when a thread wishes to put itself to sleep

• `signal()` → when a thread has changed sth. in the program and wants to wake a sleeping thread waiting on this condition.

`pthread_cond_wait(&c, &mu)` needs the mutex because `wait()` also releases the lock
`pthread_cond_signal(&c)`

`wait()` → also takes a mutex as param

→ assumes μ is locked when `wait()` is called !!

→ release lock + put thread to sleep (atomically) !

→ when thread wakes up (signaled by another thread), it must re-acquire the lock before returning to the caller.

(to prevent race cond.)

HOLD THE LOCK WHEN CALLING WAIT() / SIGNAL()

! YES !

```
void thr_exit() {  
    pthread_mutex_lock(&mu);  
    done = 1;  
    pthread_mutex_signal(&c);  
    pthread_mutex_unlock(&mu);  
}  
void thr_join() {  
    pthread_mutex_lock(&mu);  
    while (done == 0) pthread_cond_wait(&c, &mu);  
    pthread_mutex_unlock(&mu);  
}
```

! NO !

```
void thr_exit() {  
    done = 1;  
    pthread_cond_signal(&c);  
}  
void thr_join() {  
    if (done == 0) {  
        pthread_cond_wait(&c);  
    }  
}
```


Producer / Consumer!

Producers → generate data → place them in a buffer. = shared resource
Consumers → grab items → consume them from buffer

Ex: `grep foo file.txt | wc -l` ← runs 2 processes concurrently
↓
write lines with "foo" in them from file.txt to stdout → becomes input for `wc -l`
⇕
`grep` = PRODUCER ← buffer → `wc` = CONSUMER
 between them

`put()` → puts a value into the shared buffer
→ sets count = 1

`get()` → take
→ sets count = 0

{ only `put()` when count = 0
only `get()` when count = 1

producer threads
consumer threads

Locks [28]

• lock = a variable { ~~acquired~~ available
 acquired

ex: `lock_t mutex`
 ...
 `lock(&mutex)`
 `bal = bal + 1`
 `unlock(&mutex)`

← general

POSIX →

`pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;`
 ...
 `lock(&lock);`
 `bal = bal + 1;`
 ...
 `unlock(&lock);`

• in POSIX, locks = mutexes.

Jobs of the lock: 1) mutual exclusion
 2) fairness
 3) performance